Computational Analysis of Diffusive Transport

Using Numerical Differentiation and Integration

Submitted by

SANJEEV KUMAWAT

Roll Number: ph25mscst11026

M.Sc. Physics

Instructor

ANUPAM GUPTA

Date of Submission

March 18, 2026

Contents

1 Introduction **2**

2 Concentration as a Spatial Field **2**
2.1 Experimental Dataset . . . 2
2.2 Concentration Profile . . . 3

3 Local Diffusive Flux **4**
3.1 Numerical Estimation of the Concentration Gradient . . . 4
3.2 Computation of Diffusive Flux . . . 5
3.3 Results . . . 5
3.4 Discussion . . . 6

4 Integrated Quantities and Transport **6**
4.1 Total Solute Content . . . 6
4.2 Net Diffusive Transport . . . 7
4.3 Results . . . 7
4.4 Discussion . . . 8

5 Geometry of the Interface **8**
5.1 Interface Shape Dataset . . . 8
5.2 Slope of the Interface . . . 9
5.3 Arc Length Element . . . 9
5.4 Normal Vector to the Interface . . . 9
5.5 Total Flux Across the Curved Interface . . . 10
5.6 Results . . . 11
5.7 Discussion . . . 13

6 Conclusion **13**

A Python Code **14**
A.1 Concentration Profile Visualization . . . 16
A.2 Numerical Differentiation . . . 18
A.3 Numerical Integration . . . 19
A.4 Interface Geometry Analysis . . . 20

1 Introduction

Diffusion near soft, curved interfaces is central to transport in biological and physicochemical systems. The driving quantity is the spatial gradient of concentration, which governs local flux through Fick's first law, $J(x) = -D\, dc/dx$, while experiments typically demand global measures — the net flux across an interface or the total solute content of a region.

In practice, concentration fields are sampled discretely, and all gradients and integrals must be extracted numerically. The choice of scheme is consequential: differentiation amplifies noise while integration suppresses it, and the geometry of a curved interface modifies what "flux across a boundary" even means physically.

This report applies finite difference and numerical integration methods to a model diffusion problem — eleven uniformly spaced concentration measurements near a gently curved soft interface, with $D = 0.10$ cm^2/s — and demonstrates how the numerical approach shapes the physics one extracts from the data.

2 Concentration as a Spatial Field

The solute concentration $c(x)$ is measured experimentally along a line parallel to the interface at uniformly spaced spatial locations. The dataset represents discrete measurements of the concentration field along the horizontal direction. These measurements are treated as experimental data and therefore form the basis for subsequent numerical differentiation and integration.

2.1 Experimental Dataset

Index	x (cm)	$c(x)$ (arb. units)
1	0.0	1.00
2	0.5	0.94
3	1.0	0.88
4	1.5	0.81
5	2.0	0.74
6	2.5	0.69
7	3.0	0.63
8	3.5	0.59
9	4.0	0.55
10	4.5	0.52
11	5.0	0.50

Table 1: Discrete measurements of solute concentration $c(x)$ along the spatial coordinate x.

2.2 Concentration Profile

The solute concentration $c(x)$ was measured at uniformly spaced positions along the x-direction with spacing $h = 0.5\,\text{cm}$. The concentration profile $c(x)$ shows a smooth and monotonic decrease from 1.0 at $x = 0\,\text{cm}$ to 0.50 at $x = 5\,\text{cm}$.

A straight-line fit of the concentration profile was performed using the relation

$$c(x) = mx + b$$

The fitted parameters obtained from the regression analysis are

$$slope(m) = -0.104$$

$$intercept(b) = 0.9736$$

$$\Delta m : 0.00501$$

$$\Delta b = 0.0148$$

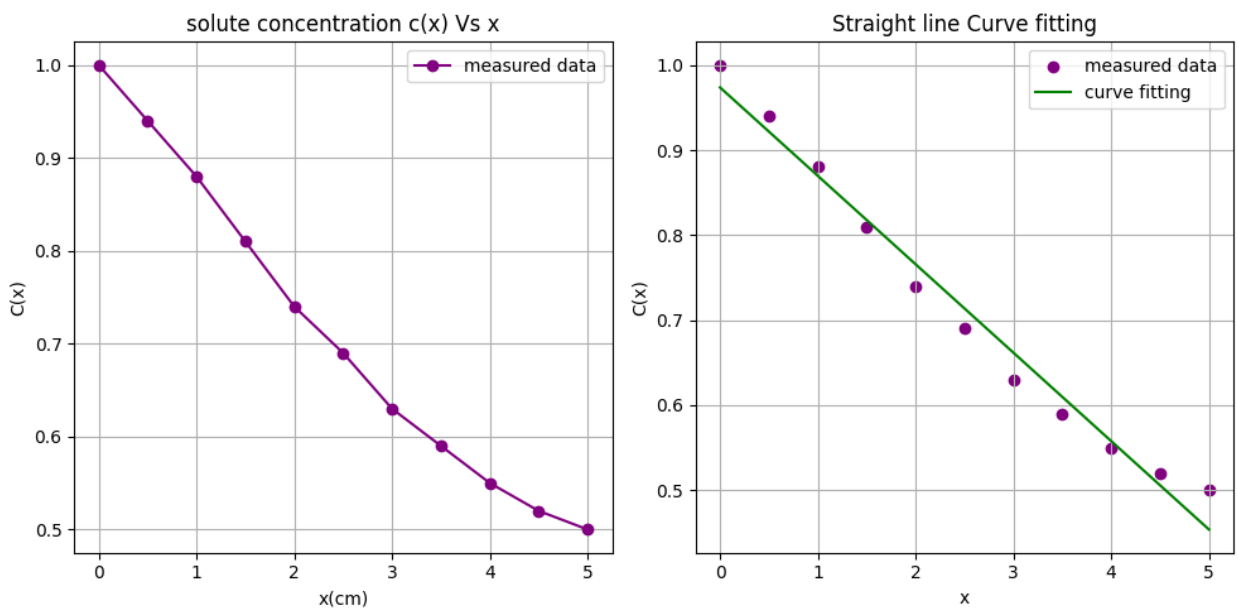

Figure 1: Concentration profile

The negative slope confirms that concentration decreases along the x-direction, which means the solute is diffusing from left to right — from high concentration toward low concentration. The small uncertainties in the fitted parameters show that a straight line describes the data well.
This kind of linear profile is physically what we expect when diffusion has reached a steady state — where the concentration no longer changes with time and varies linearly in space.

However, the gradient is not perfectly constant: it is slightly steeper near x = 0 and gentler near x = 5, which suggests the system may not be exactly at steady state.
The profile looks smooth, but this does not mean its derivative will also be smooth. As we will see in Part 3, even small variations between data points get amplified when we differentiate numerically.

3 Local Diffusive Flux

Diffusive transport in a concentration is governed by Fick's law. The diffusive flux along the spatial direction x is given by

$$J(x) = -D\frac{dc}{dx}$$

where $J(x)$ represents the local diffusive flux , D is the diffusion coefficient, and $\frac{dc}{dx}$ denotes the spatial gradient of the concentration field. For the present analysis, the diffusion coefficient is taken as

$$D = 0.10 \text{ cm}^2/\text{s}.$$

Since the concentration measurements are available only at discrete spatial points, the spatial gradient must be estimated numerically using finite difference methods.

3.1 Numerical Estimation of the Concentration Gradient

To evaluate the gradient $\frac{dc}{dx}$ from the discrete dataset, numerical differentiation schemes are employed.In this work the central, forward and backward difference methods are used.

Forward Difference

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$$

This method uses the current and next data point and has first-order accuracy, $O(\Delta x)$.

Backward Difference

$$f'(x_i) \approx \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$$

This method uses the current and previous data point and is also first-order accurate, $O(\Delta x)$.

Central Difference

$$f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1})}{x_{i+1} - x_{i-1}}$$

The central difference method uses points on both sides of x_i and provides higher accuracy, $O(\Delta x^2)$. Therefore, it is generally preferred for estimating derivatives at interior points.

3.2 Computation of Diffusive Flux

Once the concentration gradient is obtained numerically, the local diffusive flux is calculated directly using Fick's law. The resulting flux distribution provides insight into how the concentration gradient drives transport across the measured region.

3.3 Results

The spatial variation of the estimated concentration gradient and the corresponding diffusive flux are shown in the following figures.

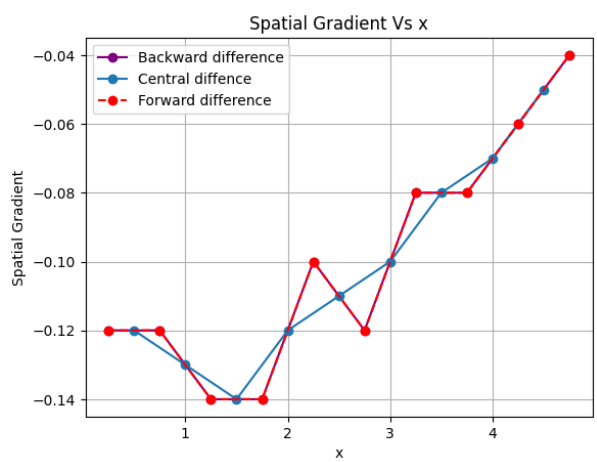

Figure 2: Numerically estimated concentration gradient $\frac{dc}{dx}$ as a function of position.

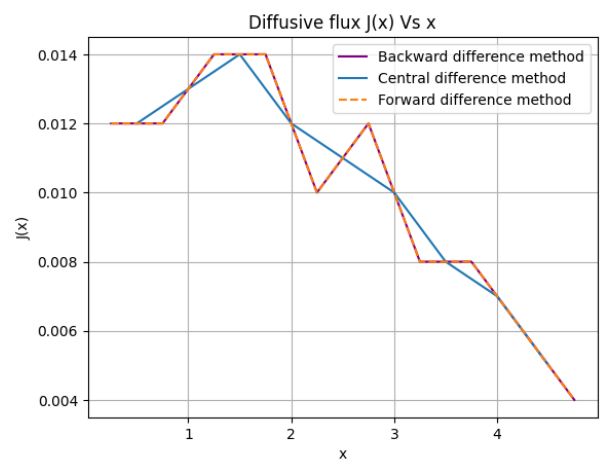

Figure 3: Local diffusive flux $J(x)$ computed using Fick's law.

Calculated spatial gradient corresponding to the spatial position -

```
Backward difference method
Spatial Gradient: [-0.12 -0.12 -0.14 -0.14 -0.1  -0.12 -0.08 -0.08
   -0.06 -0.04]
Spatial Gradient location: [0.25 0.75 1.25 1.75 2.25 2.75 3.25 3.75
   4.25 4.75]

Center difference method
Spatial Gradient: [-0.12 -0.13 -0.14 -0.12 -0.11 -0.1  -0.08 -0.07
   -0.05]
Spatial Gradient location: [0.5 1.  1.5 2.  2.5 3.  3.5 4.  4.5]

Forward difference method
Spatial Gradient: [-0.12 -0.12 -0.14 -0.14 -0.1  -0.12 -0.08 -0.08
   -0.06 -0.04]
Spatial Gradient location: [0.25 0.75 1.25 1.75 2.25 2.75 3.25 3.75
   4.25 4.75]
```

Calculated Local diffusive flux -

```
Backward difference method
Local diffusive flux: [0.012 0.012 0.014 0.014 0.01  0.012 0.008
   0.008 0.006 0.004]

Central difference method
Local diffusive flux: [0.012 0.013 0.014 0.012 0.011 0.01  0.008
   0.007 0.005]

Forward difference method
Local diffusive flux: [0.012 0.012 0.014 0.014 0.01  0.012 0.008
   0.008 0.006 0.004]
```

3.4 Discussion

The computed gradient remains negative across the spatial domain, indicating that the concentration decreases monotonically with increasing x. Consequently, the diffusive flux remains positive, consistent with the physical expectation that diffusion occurs from regions of higher concentration toward lower concentration.
Among the three schemes, the central difference method produces the smoothest flux profile because it is second-order accurate and uses points from both sides. The forward and backward methods, being first-order accurate, show more point-to-point variation in the computed flux. This is not a physical effect, it is a numerical artefact caused by the lower accuracy of one-sided schemes.

4 Integrated Quantities and Transport

While the diffusive flux computed earlier represents a local quantity, it is often useful to evaluate global transport measures by integrating the concentration and flux over the spatial domain.

4.1 Total Solute Content

The total amount of solute present in the measured region is obtained by integrating the concentration profile:

$$M = \int c(x)\, dx$$

Since the data are available only at discrete spatial points, numerical integration methods are used.
Trapezoidal Rule

$$M \approx \sum_{i=1}^{N-1} \frac{c_i + c_{i+1}}{2}(x_{i+1} - x_i)$$

Simpson's 1/3 Rule

$$M \approx \frac{h}{3}\left[c_0 + c_n + 4\sum_{\text{odd}} c_i + 2\sum_{\text{even}} c_i\right]$$

Simpson's 3/8 Rule

$$M \approx \frac{3h}{8}\left[c_0 + c_n + 3\sum_{\text{non-multiples of 3}} c_i + 2\sum_{\text{multiples of 3}} c_i\right]$$

4.2 Net Diffusive Transport

The net diffusive transport across the region is computed by integrating the diffusive flux:

$$\Phi_x = \int J(x)\,dx$$

The same numerical integration schemes can be applied to the computed flux values to estimate the total transport.

4.3 Results

Total solute content along the x direction:

```
total solute content(M):
M_trapz: 3.5500000000000003
M_simpson_1_3: 3.5499999999999994
```

Net diffusive flux along the x direction:

```
net diffusive transport for the Jx calculated by CDM:
phi_x_trapz: 0.041749999999999995
phi_x_simpson_1_3: 0.041833333333333333

net diffusive transport for the Jx calculated by BDM/FDM:
phi_x_trapz: 0.04599999999999999
phi_x_simpson_3_8: 0.04612499999999999
```

Table 2: Integrated quantities computed using different numerical integration methods

Quantity	**Trapezoidal**	**Simpson**
M (arb. units·cm)	3.5500	3.5500
Φ_x – CDM (arb. units $\cdot$ cm^2/s)	0.04175	0.04183
Φ_x – BDM (arb. units $\cdot$ cm^2/s)	0.04600	0.04612

4.4 Discussion

The total solute content $M \approx 3.55 arb.units.cm$ and the net diffusive transport 0.042 *arb. units·cm²/s* are consistent across all integration schemes- trapezoidal and Simpson rules. This close agreement confirms that the results are numerically reliable and not sensitive to the choice of integration method.
Integration is generally more numerically stable than differentiation because it averages neighbouring values rather than amplifying point-to-point fluctuations. As a result, the integrated flux Φ_x remains smooth and consistent even though the local flux $J(x)$ shows variations between the different finite-difference methods.
The small difference between the central difference method (CDM) and the backward/forward difference methods (BDM/FDM) results,

$$\Phi_x \approx 0.042 \quad \text{vs} \quad 0.046,$$

arises from the different sampling locations and accuracies of the gradient estimates. However, the integration process significantly reduces these discrepancies, leading to a stable estimate of the net transport.

5 Geometry of the Interface

In many physical systems, interfaces separating different phases are not perfectly flat but exhibit spatial curvature. The geometry of such interfaces can influence the effective transport of diffusing species. In the present case, the interface shape is described by a function $y = h(x)$, which is measured experimentally at discrete spatial locations.

5.1 Interface Shape Dataset

The experimentally measured interface profile is given in Table 3.

Index	x (cm)	$c(x)$	$h(x)$ (cm)
1	0.0	1.00	0.00
2	0.5	0.94	0.08
3	1.0	0.88	0.15
4	1.5	0.81	0.21
5	2.0	0.74	0.25
6	2.5	0.69	0.26
7	3.0	0.63	0.25
8	3.5	0.59	0.21
9	4.0	0.55	0.15
10	4.5	0.52	0.08
11	5.0	0.50	0.00

Table 3: Discrete measurements describing the interface geometry $h(x)$ along the spatial coordinate x.

5.2 Slope of the Interface

The local slope of the interface is obtained by differentiating the interface function,

$$\frac{dh}{dx}$$

Since the interface profile is available only at discrete points, the slope is estimated numerically using finite difference methods.

5.3 Arc Length Element

The arc length element associated with the curved interface is given by

$$ds = \sqrt{1 + \left(\frac{dh}{dx}\right)^2}\, dx$$

5.4 Normal Vector to the Interface

Transport across a curved interface depends on the component of the flux normal to the surface.
The interface is described by

$$y = h(x)$$

Therefore,

$$\frac{dy}{dx} = \frac{dh}{dx}$$

For a curve $y = h(x)$, the arc length element is

$$ds = \sqrt{dx^2 + dy^2}$$

Factoring out dx gives

$$ds = dx\sqrt{1 + \left(\frac{dy}{dx}\right)^2}$$

Since $y = h(x)$,

$$ds = dx\sqrt{1 + \left(\frac{dh}{dx}\right)^2}$$

Dividing both sides by ds,

$$\frac{dx}{ds} = \frac{1}{\sqrt{1 + \left(\frac{dh}{dx}\right)^2}}$$

Using the chain rule,

$$\frac{dy}{ds} = \frac{dy}{dx}\frac{dx}{ds}$$

Substituting the expressions,

$$\frac{dy}{ds} = \frac{dh/dx}{\sqrt{1 + \left(\frac{dh}{dx}\right)^2}}$$

Geometric Interpretation

The unit tangent vector to the curve is

$$\hat{\mathbf{t}} = \frac{dx}{ds}\,\hat{\mathbf{x}} + \frac{dy}{ds}\,\hat{\mathbf{y}}$$

The unit normal vector, perpendicular to the tangent, is

$$\hat{\mathbf{n}} = -\frac{dy}{ds}\,\hat{\mathbf{x}} + \frac{dx}{ds}\,\hat{\mathbf{y}}$$

These expressions are used to project the diffusive flux onto the interface normal in the numerical calculation.

5.5 Total Flux Across the Curved Interface

The total diffusive transport across the curved interface is obtained by integrating the projection of the flux vector onto the interface normal,

$$\Phi = \int \mathbf{J} \cdot \hat{n}\, ds$$

Since unit normal vector is

$$\hat{\mathbf{n}} = -\frac{dh/dx}{\sqrt{1+(dh/dx)^2}}\,\hat{\mathbf{x}} + \frac{1}{\sqrt{1+(dh/dx)^2}}\,\hat{\mathbf{y}}$$

In the given problem diffusive flux is directed along the x-axis,

$$\mathbf{J} = J_x\hat{\mathbf{x}},$$

the projection of the flux onto the interface normal becomes

$$\mathbf{J}\cdot\hat{\mathbf{n}} = J_x\left(-\frac{dh/dx}{\sqrt{1+(dh/dx)^2}}\right)$$

Multiplying by the arc-length element

$$ds = \sqrt{1+(dh/dx)^2}\,dx,$$

we obtain

$$\mathbf{J}\cdot\hat{\mathbf{n}}\,ds = -\,J_x\,\frac{dh}{dx}\,dx$$

$$\Phi = \int -\,J_x\,\frac{dh}{dx}\,dx$$

5.6 Results

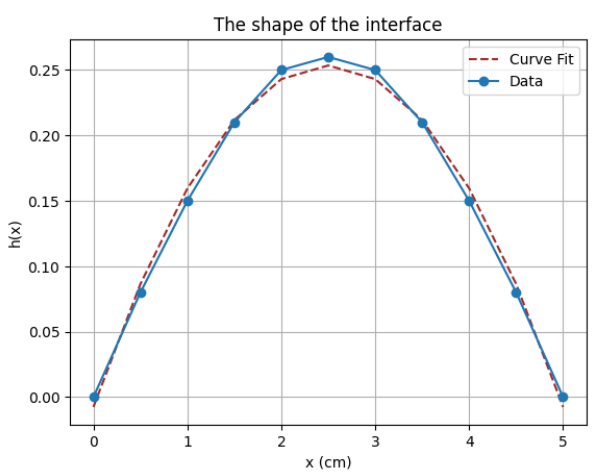

Figure 4: Geometry $h(x)$ along the spatial coordinate x.

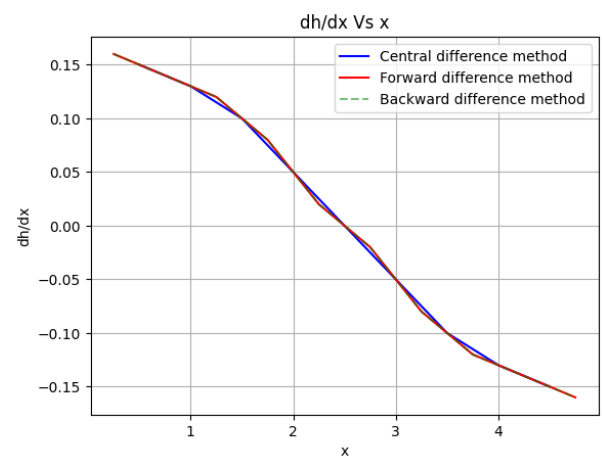

Figure 5: Numerically estimated slope dh/dx of the interface

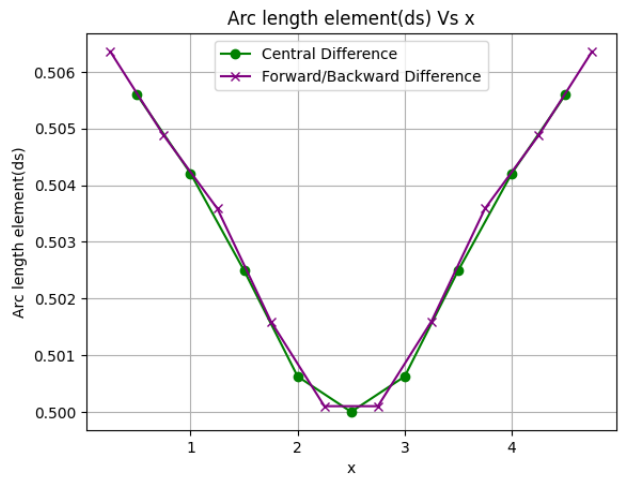

Figure 6: Arc length element corresponding x

Values of the slope of the curved Interface calculated using Central, Forward and Backward difference method:

```
dh/dx(Central difference method): [ 0.15  0.13  0.1   0.05  0.
   -0.05 -0.1  -0.13 -0.15]
dh/dx(Forward difference method): [ 0.16  0.14  0.12  0.08  0.02
   -0.02 -0.08 -0.12 -0.14 -0.16]
dh/dx(Backward difference method): [ 0.16  0.14  0.12  0.08  0.02
   -0.02 -0.08 -0.12 -0.14 -0.16]
```

Estimated values of the Arc length:

```
Arc length element(ds)(central difference method):
 [0.50559371 0.5042073  0.50249378 0.50062461 0.5        0.50062461
 0.50249378 0.5042073  0.50559371]

Arc length element(ds)(forward/backward difference method):
 [0.50635956 0.50487622 0.50358713 0.50159745 0.50009999 0.50009999
 0.50159745 0.50358713 0.50487622 0.50635956]
```

Estimated values of the total flux through the curved surface:

```
Calculation using trapezoidal method
The total flux across the interface by central difference method:
   -0.0010024999999999995
The total flux across the interface by backward difference method:
   -0.00132

Calculation using Simpson rule
The total flux(CDM): -0.0009616666666666668
```

```
The total flux(BDM): -0.0012749999999999997
```

The total flux across the flat interface:

$$\Phi = 0$$

Since the angle between the unit normal of a flat interface and the diffusive flux J is 90°, their dot product is zero, giving $\Phi = 0$

Table 4: Comparison of numerical integration methods for total flux

Method	**Trapezoidal**	**Simpson**
Central difference	-0.00100	-0.000962
Backward difference	-0.00132	-0.001275
Flat interface	0	0

5.7 Discussion

The total flux across the curved interface is

$$\Phi \approx -0.001,$$

which is very close to zero. This result is not a numerical error but arises from the symmetric arch shape of $h(x)$. On the left half of the interface the slope is positive, producing a negative contribution to the flux integral, while on the right half the slope is negative, producing a positive contribution.
These contributions nearly cancel each other, resulting in a very small net flux. Thus, the curvature itself does not enhance transport; rather, the symmetry of the interface leads to cancellation of opposite contributions. For an asymmetric interface, such cancellation would not occur and the total flux would generally be non-zero.

6 Conclusion

In this work, Numerical differentiation and integration were employed to analyze diffusive transport near a curved interface. The concentration profile was smooth and nearly linear, indicating steady diffusion. Numerical differentiation revealed noticeable fluctuations in the gradient estimates, with the central difference method providing the most reliable results.
In contrast, numerical integration produced stable global quantities, yielding a total solute content

$$M \approx 3.55 \text{ arb. units} \cdot \text{cm}$$

and a net transport

$$\Phi_x \approx 0.042.$$

When the interface geometry was included, the total flux across the curved surface was found to be

$$\Phi \approx -0.001,$$

which is essentially zero. This occurs because the symmetric interface produces equal and opposite contributions that cancel within the integral.
Overall, the results demonstrate that numerical differentiation tends to amplify noise, whereas numerical integration suppresses it. Furthermore, the geometry of the interface plays a key role in determining how diffusive transport is interpreted.

A Python Code

Imports

```
#importing required libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
```

Numerical Methods

```
#Central Difference method
def central_difference_method(x, y):
    derivative = []
    x_loc = []

    for i in range(1, len(x)-1):
        dy = (y[i+1] - y[i-1])/(x[i+1] - x[i-1])
        derivative.append(dy)
        x_loc.append(x[i])

    return np.array(derivative), np.array(x_loc)


#Forward Difference method
def forward_difference_method(x,y):
    gradient = []
    x_loc = []

    for i in range(len(x)-1):
        g = (y[i+1] - y[i])/(x[i+1] - x[i])
        gradient.append(g)
        x_loc.append((x[i] + x[i+1])/2)

    return np.array(gradient), np.array(x_loc)
```

```
#Backward difference method
def backward_difference_method(x,y):
    gradient = []
    x_loc = []

    for i in range(1, len(x)):
      g = (y[i] - y[i-1])/(x[i] - x[i-1])
      gradient.append(g)
      x_loc.append((x[i] + x[i-1])/2)

    return np.array(gradient), np.array(x_loc)

#trapezoidal method

def trapezoidal_method(x, y):
  int_value = 0.0
  for i in range(len(x) - 1):
    int_value +=  (x[i+1] - x[i]) * (y[i] + y[i+1])/2
  return int_value

#simpson 1/3rd Rule Formula

def simpson_1_3_rule(x, y):

    n = len(x)

    if (n-1) % 2 != 0:
        raise ValueError("Simpson 1/3 rule requires even number of
   intervals")

    h = x[1] - x[0]

    odd_term = 0.0
    even_term = 0.0

    for i in range(1, n-1, 2):
        odd_term += y[i]

    for i in range(2, n-1, 2):
        even_term += y[i]

    int_value = (h/3) * (y[0] + y[-1] + 4*odd_term + 2*even_term)

    return int_value


#simpson 3/8th rule
```

```
def simpson_3_8_rule(x, y):

    n = len(x)

    if (n-1) % 3 != 0:
        raise ValueError("Simpson 3/8 rule requires intervals
   multiple of 3")

    h = x[1] - x[0]

    multiple_of_3 = 0.0
    other_term = 0.0

    for i in range(1, n-1):
        if i % 3 == 0:
            multiple_of_3 += y[i]
        else:
            other_term += y[i]

    int_value = (3*h/8) * (y[0] + y[-1] + 3*other_term + 2*
   multiple_of_3)

    return int_value
```

Formulae:

```
#arc length
def arc_length(x, y):
    dydx, x_loc = central_difference_method(x, y)
    dx = x[1] - x[0]
    ds = dx * np.sqrt(1 + dydx**2)
    return ds, x_loc

#unit normal vector
def unit_normal_vector(dydx):
    denom = np.sqrt(1 + dydx**2)
    nx = -dydx / denom
    ny = 1 / denom
    return nx, ny
```

A.1 Concentration Profile Visualization

```
#Measurements
x = np.array([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5,
   5.0])  #(cm)
cx = np.array([1.00, 0.94, 0.88, 0.81, 0.74, 0.69, 0.63, 0.59, 0.55,
    0.52, 0.50]) #(arb. units)
```

```
#curve fitting
def straight_line(x, m, c):
  return m*x + c

popt, pcov = curve_fit(straight_line, x, cx)
#intercepts
m, c = popt
#error in parameters
sigma_m, sigma_c = np.sqrt(np.diag(pcov))

#fitted line
cx_fit = straight_line(x, m, c)

#Graphs plot
fig, ax = plt.subplots(1, 2, figsize=(10,5))

#graph for solute concentration c(x) vs x
ax[0].plot(x, cx, label="measured data", marker="o", color="purple")
ax[0].set_title("solute concentration c(x) Vs x")
ax[0].set_xlabel("x(cm)")
ax[0].set_ylabel("C(x)")
ax[0].legend()
ax[0].grid(True)

#curve fitting(straight_line)
ax[1].scatter(x, cx, label="measured data", c="purple")
ax[1].plot(x, cx_fit, label="curve fitting", color='green')
ax[1].set_title("Straight line Curve fitting")
ax[1].set_xlabel("x")
ax[1].set_ylabel("C(x)")
ax[1].legend()
ax[1].grid(True)

#tight layout
plt.tight_layout()
plt.show()

print("Results of Straight line fit")
print(f"Slope: {m}")
print(f"Intercept: {c}")

#error in slope and intercept
print(f'error in slope: {sigma_m}')
print(f'error in intercept: {sigma_c}')
```

A.2 Numerical Differentiation

```
#calculating gradient using backward, central, forward difference
   method
gradient_bdm, x_loc_bdm = backward_difference_method(x, cx)
gradient_cdm, x_loc_cdm = central_difference_method(x, cx)
gradient_fdm, x_loc_fdm = forward_difference_method(x, cx)


plt.plot(x_loc_bdm, gradient_bdm, color="purple", label="Backward
   difference", marker= "o")
plt.plot(x_loc_cdm, gradient_cdm, label="Central diffence", marker=
   "o")
plt.plot(x_loc_fdm, gradient_fdm, label="Forward difference",
   linestyle="--", marker= "o", color="r")

plt.title("Spatial Gradient Vs x")
plt.xlabel("x")
plt.ylabel("Spatial Gradient")
plt.legend()
plt.grid(True)
plt.show()


#print the results

#backward difference method
print("Backward difference method")
print(f"Spatial Gradient: {gradient_bdm}")
print(f"Spatial Gradient location: {x_loc_bdm}\n")

#centeral difference method
print("Centeral difference method")
print(f"Spatial Gradient: {gradient_cdm}")
print(f"Spatial Gradient location: {x_loc_cdm}\n")

#Forward difference method
print("Forward difference method")
print(f"Spatial Gradient: {gradient_fdm}")
print(f"Spatial Gradient location: {x_loc_fdm}\n")



#stepsize
h = 0.5

#value of Diffusion coeffiecient
D = 0.10 #cm^2/s
```

```
#Local diffusive flux
def J(D, cx_dx):
  return -D*cx_dx

Jx_backward = J(D, gradient_bdm)
Jx_central = J(D, gradient_cdm)
Jx_forward = J(D, gradient_fdm)


#plot for diffusive flux
plt.plot(x_loc_bdm,Jx_backward, label="Backward difference method",
   color="purple")
plt.plot(x_loc_cdm,Jx_central, label="Central difference method")
plt.plot(x_loc_fdm,Jx_forward, label="Forward difference method",
   linestyle="--")
plt.title("Diffusive flux J(x) Vs x")
plt.xlabel("x")
plt.ylabel("J(x)")
plt.legend(loc="best")
plt.grid(True)
plt.show()


#print the results

print(f"Backward difference method")
print(f"Local diffusive flux: {Jx_backward}\n")

print(f"Central difference method")
print(f"Local diffusive flux: {Jx_central}\n")

print(f"Forward difference method")
print(f"Local diffusive flux: {Jx_forward}\n")
```

A.3 Numerical Integration

```
#Integration using trapezoidal method and Simpson rule

#Compute total solute content(M)
M_trapz = trapezoidal_method(x,cx)
M_simpson_1_3 = simpson_1_3_rule(x,cx)
#M_simpson_3_8 = simpson_3_8_rule(x,cx)

#net diffusive transport for the Jx calculated by CDM
phi_x_trapz_cdm =  trapezoidal_method(x_loc_cdm, Jx_central )
phi_x_simpson_1_3 = simpson_1_3_rule(x_loc_cdm, Jx_central)
```

```
#net diffusive transport for the Jx calculated by BDM/FDM
phi_x_trapz_bdm =  trapezoidal_method(x_loc_bdm, Jx_backward)
phi_x_simpson_3_8 = simpson_3_8_rule(x_loc_bdm, Jx_backward)

#print results
print("total solute content(M):")
print(f"M_trapz: {M_trapz}")
print(f"M_simpson_1_3: {M_simpson_1_3} \n")

print("net diffusive transport for the Jx calculated by CDM:")
print(f"phi_x_trapz: {phi_x_trapz_cdm}")
print(f"phi_x_simpson_1_3: {phi_x_simpson_1_3}\n")

print("net diffusive transport for the Jx calculated by BDM/FDM:")
print(f"phi_x_trapz: {phi_x_trapz_bdm}")
print(f"phi_x_simpson_3_8: {phi_x_simpson_3_8}")
```

A.4 Interface Geometry Analysis

```
#interface geometry measurement

hx = np.array([0, 0.08, 0.15, 0.21, 0.25, 0.26, 0.25, 0.21,  0.15,
   0.08, 0])


def quadratic(x, a, b, c):
    return a*x**2 + b*x + c

popt, pcov = curve_fit(quadratic, x, hx)

# intercepts
a, b, c = popt
# curve fitting
plt.plot(x, quadratic(x, a, b, c), label="Curve Fit", color='brown',
    linestyle="--")
# shape of the interface
plt.plot(x, hx, marker="o", label="Data")
plt.title("The shape of the interface")
plt.xlabel("x (cm)")
plt.ylabel("h(x)")
plt.grid(True)
plt.legend()
plt.show()

# 1)
#differentian using central difference method and FDM/BDM
```

```
dh_cdm, x_loc_cdm = central_difference_method(x, hx)
dh_fdm, x_loc_fdm = forward_difference_method(x, hx)
dh_bdm, x_loc_bdm = backward_difference_method(x, hx)

#plot
plt.plot(x_loc_cdm, dh_cdm, label="Central difference method", color
   ="blue")
plt.plot(x_loc_fdm, dh_fdm, label="Forward difference method", color
   ="red")
plt.plot(x_loc_bdm, dh_bdm, label="Backward difference method",
   color="green", linestyle="--", alpha=0.5)
plt.title("dh/dx Vs x")
plt.xlabel("x")
plt.ylabel("dh/dx")
plt.grid(True)
plt.legend()
plt.show()

#print the results
print(f"\ndh/dx(Central difference method): {dh_cdm}")
print(f"dh/dx(Forward difference method): {dh_fdm}")
print(f"dh/dx(Backward difference method): {dh_bdm}")

#2) Arc length element
dx = h #0.5

#central difference method
dh_square_cdm = np.square(dh_cdm)
ds_cdm = dx*np.sqrt(1 + dh_square_cdm)

#backward/forward difference method
dh_square_bdm = np.square(dh_bdm)
ds_bdm = dx*np.sqrt(1 + dh_square_bdm)

#plots
plt.plot(x_loc_cdm, ds_cdm, label="Central Difference",  color="
   green")
plt.plot(x_loc_bdm, ds_bdm, label="Forward/Backward Difference",
   color="purple")
plt.title("Arc length element(ds) Vs x")
plt.xlabel("x")
plt.ylabel("Arc length element(ds)")
plt.legend(loc="best")
plt.grid(True)
plt.show()

#Print the results
```

```
print(f"Arc length element(ds)(central difference method): \n {
   ds_cdm}\n")
print(f"Arc length element(ds)(forward/backward difference method):
   \n {ds_bdm}")



#Calculations using central difference method
dh_cdm, x_loc_cdm = central_difference_method(x, hx)
# ds_cdm = dx * np.sqrt(1 + dh_square_cdm)
dxds_cdm = 1/np.sqrt(1 + dh_square_cdm)
dyds_cdm = dh_cdm/np.sqrt(1 + dh_square_cdm)
Jx_central = J(D, gradient_cdm)
#unit normal components
# nx_cdm = -dyds_cdm
# ny_cdm = dxds_cdm
J_dot_n_cdm = Jx_central * (-dh_cdm) #Jx_central * nx_cdm
J_globally_cdm = trapezoidal_method(x_loc_cdm, J_dot_n_cdm)
J_globally_cdm_simpson_1_3 = simpson_1_3_rule(x_loc_cdm, J_dot_n_cdm
   )

#Calculations using Forward/Backward difference method
dh_bdm, x_loc_bdm= backward_difference_method(x, hx)
# ds_bdm = dx * np.sqrt(1 + dh_square_bdm)
dxds_bdm = 1/np.sqrt(1 + dh_square_bdm)
dyds_bdm = dh_bdm/np.sqrt(1 + dh_square_bdm)
Jx_backward = J(D, gradient_bdm)
#unit normal components
nx_bdm = -dyds_bdm
ny_bdm = dxds_bdm
J_dot_n_bdm = Jx_backward * (-dh_bdm)
J_globally_bdm = trapezoidal_method(x_loc_bdm, J_dot_n_bdm)
J_globally_bdm_simpson_3_8 = simpson_3_8_rule(x_loc_bdm, J_dot_n_bdm
   )


#print the results
print("Calculation using trapezoidal method")
#The total flux across the interface by central difference method
print(f"The total flux across the interface by central difference
   method: {J_globally_cdm}")

#The total flux across the interface by backward difference method
print(f"The total flux across the interface by backward difference
   method: {J_globally_bdm}\n")

print("Calculation using Simpson rule")
```

```
print(f"The total flux: {J_globally_cdm_simpson_1_3}")
print(f"The total flux: {J_globally_bdm_simpson_3_8}")


#for the flat interface
dhdx = 0
nx, ny = unit_normal_vector(dhdx)
J_flat = J(D, gradient_cdm)
J_dot_n_flat = J_flat * nx   # nx = 0
J_globally_flat = trapezoidal_method(x_loc_cdm, J_dot_n_flat)

print(f"The total flux across the flat interface: {J_globally_flat}"
   )
```

References

[1] Khan Academy, "Constructing a Unit Normal Vector to a Curve," Multivariable Calculus — Integrating Multivariable Functions. Available at: https://www.khanacademy.org/math/multivariable-calculus/integrating-multivariable-functions/line-integrals-in-vector-fields-articles/a/constructing-a-unit-normal-vector-to-curve [Accessed: 17 March 2026].

[2] Sanjeev K, "Python code for Numerical Differentiation and Integration used in this report," GitHub repository, 2026. Available at: https://github.com/arsanjeevk/Computational_Methods/tree/main/Assignment-7(Differentiation%20&%20Integration) [Accessed: 17 March 2026].